4

Terraform Commands and State Management

In the previous chapters, you learned about the building blocks of the Terraform configuration language and the out-of-the-box capabilities it provides, such as multi-cloud support, extensive support for different providers, and state management. This chapter is all about the various commands you will use while working with Terraform CLI.

Each command has its own significance and purpose, and you may not use everything in regular scenarios. But there are a few commands (such as `terraform init`, `terraform apply`, and `terraform destroy`) that will be a part of almost every infrastructure workflow and are used as and when the need arises.

The following exam objectives will be covered in this chapter:

- Use the core Terraform workflow
- Implement and maintain state

You will start by looking at the aforementioned primary commands and then you will cover the various subcommands. This will then be followed by concepts around Terraform state management.

This chapter will cover the following topics:

- Basic workflow commands
- Commands for code management
- Special commands
- State management and the `terraform state` command and subcommands

Technical Requirements

In this chapter, you will deep dive into Terraform commands and try them out on Terraform CLI, installed on your workstation. Please ensure that you have the following installed and ready to use:

- An AWS account ID with administrator access credentials
- AWS CLI version 2.x.x
- Terraform CLI version 1.5.x or later
- Visual Studio Code or any text editor

The GitHub URL for the chapter will contain graphics and the sample scripts used in the chapter and can be referred to at the following link:

`https://github.com/PacktPublishing/Hashicorp-Certified-Terraform-Associate-003-Exam-guide-Second-Edition/tree/main/ch4/terraform-commands-and-state-management`

Basic Workflow Commands

In the previous chapter on Terraform basics, you used the following commands in the sample exercise to create an IAM user. You saw how it helps in different phases of project development. You will now take a closer look at the basic commands. The different options available to use are listed here and will be covered in detail in the subsequent sections:

- `init`
- `plan`
- `apply`
- `destroy`

Please note that all the preceding commands will be preceded by the keyword `terraform` when you actually use them in Terraform CLI. For every command, you will be look at the command syntax and options, followed by more details on the command usage in different scenarios.

As a general rule, the command syntax will be as follows:

```
$ terraform <command_name> <command_options>
```

The init Command

As you have seen in the previous chapter on Terraform basics, the `terraform init` command is the first command that should be run to initialize the working directory where the configuration scripts are stored. The `init` command will download all the required providers and modules needed to run the script.

Based on the configuration scripts, the `init` command will perform different activities in the background, but it is safe to say that you do not need to know the details. Suppose the provider or the module version is changed in the script; the `init` command needs to be rerun to allow the changes to take effect before the resource creation.

This is how the command is used:

```
$ terraform init <init_options>
```

Now take a look at some common options supported by the `init` command:

- `-input=true`: If you are setting this option, it will ask for an input to execute the `init` command if necessary. If this is set to `false`, the command will fail directly without the user prompt. The default value is `true`.

  This option is like using Amazon S3 as the backend to store the `terraform state` file and the partial configuration (not all inputs are provided for successful backend configuration) is used in the script.

  A backend for S3 would require a bucket name, region, and key name to store the file. If any of the inputs are missing, the default behavior is that it will be prompted when you run the `terraform init` command.

  If this option is set to `false`, the command will error out immediately.

- `-lock=false`: This option will disable the locking of state files; locking is enabled by default.
- `-lock-timeout=<duration>`: This option will override the duration of the state lock with the user-provided value; the default is zero seconds. It might help in scenarios where there are parallel processes running pointing to the same state file and it makes sense to wait for the lock to get released for the next process to use it.
- `-no-color`: This option will disable color codes in the output.
- `-upgrade`: This option will upgrade modules and providers to the latest version if applicable. If this option is used, the recorded versions in the dependency lock file will be ignored and the relevant modules and provider plugins will be upgraded to match the version constraints currently in the configuration.

- `-from-module=<module_location>`: This option will be used in special scenarios where your configuration scripts refer to a module that is not present in the local/current working directory.

  In this case, the module source location should be provided when running the `init` command, and this option helps achieve providing the module source location. Take a look at a scenario.

  You have created a new Terraform configuration and it has a module block that refers to the code available in GitHub and not available on your local machine. When you run the `init` command with the `-from-module` option, in this case, the given module will be copied to the local working directory before the initialization steps are performed.

- `-reconfigure`: If the Terraform script is configured with the `backend` block, the `init` command execution will configure the backend settings accordingly. However, there might be cases where the current backend configuration is not relevant, and changes have been made to use a different backend. This is when this option needs to be specified along with the `terraform init` command, once again, to reflect the latest changes to use the new backend.

- `-migrate-state`: This option is used to migrate the state file to the new backend, and, based on the setup, there will be additional prompts for the user to enter values to complete the state migration.

- `-force-copy`: The use case for this option is exactly the same as the `-migrate-state` option but the difference is that there will not be prompts and `yes` will be automatically assumed for state migration.

- `-backend=false`: This option is used to skip the backend configuration with the current execution of the `init` command.

- `-plugin-dir=<plugin_location>`: This option is used to provide the alternate path to search for plugins, and is primarily used in the development stages when you create new modules or develop a new provider plugin.

- `-get=false`: This option will disable downloading the modules referenced.

- `-lockfile=MODE` : This is used to set the dependency lock file mode; the only value supported now is `readonly`.

- `-lock=false`: This option will not hold the state file during the backend migration, and should be used very cautiously, especially when there will be other developers who can run the same configuration referring to the same workspace. State locking is enabled by default.

The plan Command

You now have a better idea about using the `terraform init` command that primarily deals with the initialization of the working directory. You can continue with the commands that form the core part of the Terraform provisioning workflow. The `terraform plan` command is the first one before the `apply` and `destroy` commands.

The main purpose of the `plan` command is to evaluate the configuration scripts and determine the desired state of the resources declared. Then, it will compare the desired state with the state of the real infrastructure objects using the state file and figure out the changes to achieve the desired state.

Once the changes needed are figured out, the `plan` command presents the execution plan. Please note that the `plan` command does not make any changes to real-world infrastructure objects. The responsibility of making actual changes is given to the `terraform apply` command, which usually follows next.

The plan output from the command can also be saved with the `-out` option, which can be directly given as input to the `apply` command to proceed with making the changes.

In some cases, it is also possible that the real-world infrastructure objects are in sync with the desired state of the declared resources, which means no change is needed. If you run the `terraform plan` command in this case, the command output will clearly indicate that no infrastructure change is needed.

This is how the command is used:

```
$ terraform plan <plan_options>
```

Now you can take a look at some of the options supported by the `plan` command:

- `-destroy`: This option with the `plan` command is to clean up all the resources provisioned so far, and this will result in an empty state file. The `plan` command with the `-destroy` option will exhibit the same behavior as running the `terraform destroy` command, which will be discussed in the subsequent sections.
- `-refresh-only`: This option has two purposes: updating the Terraform state file and output values from the root module to sync with the changes made outside Terraform.

  Now, when might this be useful? Assume that you have created a resource in AWS such as an IAM user or RDS database instance, initially using Terraform, and the `terraform.tfstate` state file has been updated. You also went ahead and made changes directly on the AWS Management Console.

  It is important to sync the state file with the changes made from the console, and that is when the `-refresh-only` option can be used.

- `-refresh=false`: This option will disable the refresh of the Terraform state file before applying the configuration changes. In the default scenario, the `plan` command will also check the remote objects and refresh the Terraform state to figure out the changes needed to achieve the desired state. This indirectly means Terraform will issue API calls to the respective providers and get the details, which can consume more time.

  With this option, the refresh part is disabled, and hence, the plan will run faster; this can be considered only in special scenarios because the state file can go out of sync.

  This option cannot be used along with the `-refresh-only` option since the refresh behavior will not happen at all.

- `-target=<RESOURCE_ADDRESS>`: This option is used to target particular resources or a set of resources for planning rather than for the entire configuration. This will be helpful to provision the resources that you would like to test first before completing the entire development.

- `-replace=<RESOURCE_ADDRESS>`: This option is used to replace the resources that match the given address; this is similar to the target option to focus on a particular resource or a set of resources that have been manipulated and need replacement. The earlier version of Terraform had a separate command, `terraform taint`, for this purpose, but it is deprecated now.

- `-var "VARIABLE_NAME=VALUE"`: Use this option to set a value for the variable declared in the main configuration. If there are multiple variables, the option can be used for each variable name.

  Here is a sample command:

  ```
  $ terraform plan -var "region=us-east-1"
  ```

- `-var-file=<FILENAME>`: This option is used to set the values of multiple input variables usually using the `*.tfvars` file. The option can also be used multiple times to pick the values from more than one file.

- `-detailed-exitcode`: This option changes the exit codes and meanings to provide more information on the output plan. The possible values are zero (0), one (1), and two (2) according to the execution result:

  `exitcode 0`: Success with no changes in the plan

  `exitcode 1`: Error condition

  `exitcode 2`: Success with changes in the plan

- `-generate-config-out=<PATH>`: If there are `import` blocks available in the configuration, this option will help generate Terraform resource blocks in the HCL language in the path specified. The concept of using `import` blocks is an advanced topic and will be covered in later chapters when applicable.

- `-input=false`: This option exactly works the same as we have seen with the `init` command, and it disables the default prompt for any input variable with the value missing.
- `-lock=false`: This disables the state file lock operation, which is not generally recommended if there is a possibility of multiple concurrent runs.
- `-json`: This can be set to enable the JSON formatted output and is very helpful when the Terraform execution happens in an automated fashion and the JSON format is machine readable.
- `-lock-timeout=<duration>`: This option will override the duration of the state lock with the user-provided value; the default is zero seconds. It might help in scenarios where there are parallel processes running pointing to the same state file and it makes sense to wait for the lock to get released for the next process to use it.
- `-no-color`: This option is used to disable color codes in the output.
- `-parallelism=<COUNT>`: This option can be used to override the maximum number of parallel operations on the machine running Terraform. This can be used when you want to intentionally decrease/increase the load on the host machine to have better control. The default value is ten (10).
- `-out=<FILENAME>`: As already mentioned earlier, the `terraform plan` command can also include the `-out` option to save the plan output, which can be used directly with `terraform apply`.

  One important thing to note here is that Terraform can accept any filename, but the file should not have any suffix or file extension. If a suffix is present, Terraform will consider it as a different file format and the `.tf` suffix will also result in Terraform treating it as a regular configuration file.

  Here is a sample command:

  ```
  $ terraform plan -out=tfplan -input=false
  ```

Some of these options are also supported by the `terraform apply` command, and you will read about this in the next section.

The apply Command

In the core workflow, the next command is the `apply` command and its responsibility is to make the changes proposed in the Terraform execution plan. The `terraform apply` command has two ways of operating: auto plan mode and saved plan mode.

In the auto plan mode, you cannot expect the `terraform plan` file to be saved already, and when the `apply` command is executed, it automatically creates an execution plan for you. This will be followed by a user prompt to confirm the plan and take the necessary actions. In automation cases, the prompt can be disabled with the `-auto-approve` option.

In the case of the saved plan mode, `terraform plan` was executed previously, and you have the plan already. In this mode, the `apply` operation will run and it does not need any user inputs. In this mode, additional plan options cannot be specified.

This is how the command is used:

```
$ terraform apply <apply_options>
```

The following options are also supported in the `terraform apply` command. Since you have already read about this in the previous topic, the options are just listed here:

- `-destroy`
- `-refresh-only`
- `-refresh=false`
- `-replace=<RESOURCE_ADDRESS>`
- `-target=<RESOURCE_ADDRESS>`
- `-var "VARIABLE_NAME=VALUE"`
- `-var-file=<FILENAME>`
- `-input=false`
- `-lock=false`
- `-lock-timeout=<DURATION>`
- `-parallelism=n`
- `-no-color`

Please remember that all the preceding options are applicable when the `apply` command is executed without a saved plan file.

Other options supported for `apply` are as follows, and the last three options are applicable when using the local backend:

- `-auto-approve`: Setting this option will allow the process to proceed without the user prompt to provision the resources and update the state file.
- `-compact-warnings`: This option will compact the warning messages.
- `-state=<PATH>`: This option is used to read and save the state file path; if not set, this will default to the `terraform.tfstate` file in the current working directory.

- `-state-out=<PATH>`: This option is used to save the state file in a different location, so the old state is not impacted by the latest execution.
- `-backup=<PATH>`: This is used to back up the state file in the specified path before modifying the state file.

The destroy Command

The `destroy` command is used to destroy all the real-world infrastructure objects managed by a particular configuration. As we have seen in this chapter, the `destroy` command will have the same behavior as the `terraform apply` command with the `-destroy` option. So, most of the options supported by the `apply` command can also be used with `terraform destroy`.

This is how the command is used:

```
$ terraform destroy <destroy_options>
```

Before the actual execution of `destroy`, you can also review the plan by running the following command:

```
$ terraform plan -destroy
```

This is helpful in scenarios where the applied configuration will not be used further, and the resources provisioned can be cleaned up.

Commands for Code Management

In this section, you will quickly look at a couple of commands – `terraform fmt` and `terraform validate` – used in the development stages. There will be no harm to the configuration if these are not used. The intention, however, is to check that it is the right code and is formatted properly, and to check whether there are syntactical errors.

The fmt Command

The `terraform fmt` command is used to rewrite/update the Terraform configuration scripts to a canonical format and style. This is based on the style conventions opinionated of HashiCorp itself to ensure consistency across all the files.

For example, you can generate a Terraform configuration using the `-generate-config-out` option with the `plan` command and the resulting configuration will follow the Terraform styling conventions. You also might write scripts manually for other resources, but using the `terraform fmt` command will ensure that the code is formatted consistently.

At the end of the day, it is up to the developer to choose the styling conventions, and there is no problem with the approach. It is not enforced by any means. There will be no change in the result, irrespective of the formatting approach you use.

This is how the command is used:

```
$ terraform fmt <fmt_options> <TARGET>
```

Running this command will format the files in the current working directory by default. If the `<TARGET>` option is used, the command will scan the file/directory passed in the input. If a hyphen is specified for `<TARGET>`, the command will read the input from standard input (`STDIN`).

The following are the options supported by this command and it is not mandatory to use them. Just running the `terraform fmt` command will also work if you expect the default behavior:

- `-list=false`: Setting this option will not list the files that are not properly formatted.
- `-check`: This option will just check whether the input files are formatted as per the styling conventions and the exit code will be set accordingly. It will be zero (0) if all input files are formatted properly and non-zero if there are any inconsistencies.
- `-rewrite=false`: This option will not overwrite the input files and can be used along with the `-check` option if you are just going to validate the configuration or input files.
- `-diff`: This option lists differences in the formatting changes.
- `-recursive`: This option is used to format the input files in the current working directory as well as in subdirectories. The default behavior is that the command will format the files in the current working directory only.

Take a look at a simple example of how `terraform fmt` will reorganize a simple resource block that will create an Amazon S3 (AWS storage service to store files as objects) bucket. You will notice that the `Name` key and value alignment are formatted according to the other tag in the `tags { }` inner block.

This is before running the `terraform fmt` command:

```
resource "aws_s3_bucket" "testbucket" {
  bucket = "fmt-test-bucket"

  tags = {
    Name = "Test bucket"
    Environment = "testing"
  }
}
```

This is after running the `terraform fmt` command:

```
resource "aws_s3_bucket" "testbucket" {
  bucket = "fmt-test-bucket"

  tags = {
    Name        = "Test bucket"
```

```
    Environment = "testing"
  }
}
There are few additional
```

The validate Command

The `terraform validate` command is used to validate the configuration files in the directory, checking whether the configuration is syntactically correct and that there are no inconsistencies with the variable names or any attributes. This command does not access the state file or call the provider APIs, and it works within the directory only.

The command requires the current working directory to be initialized with the `terraform init` command.

This is how the command is used:

```
$ terraform validate <validate_options>
```

The following are the options supported:

- `-json`: This option produces the machine-readable JSON output and this can help with automation scenarios.
- `-no-color`: The output will not have any color if this option is specified.

Special Commands

So far, you have read about the core Terraform workflow commands and then followed this up with the code reformatting commands that will be used in most cases. In this section, you will see some special commands in appropriate scenarios.

The login Command

The `terraform login` command is used to obtain and save an API token for HCP Terraform, Terraform Enterprise, or any other compatible host. In the case of HCP Terraform and Terraform Enterprise, the user will be interacting with the host with the API token as the authentication mechanism and it is applicable to use in interactive scenarios.

This is how the command is used:

```
$ terraform login <HOSTNAME>
```

In case `HOSTNAME` is not specified, it will default to the HCP Terraform host at `app.terraform.io`.

If the command execution is successful in CLI, the API token retrieved will be stored locally in the file named `credentials.tfrc.json` by default. But there is also an option to change this behavior and save the API token in a different location when you run the `login` command.

The logout Command

The `terraform logout` command is used to remove the API token stored by the `login` command. With this command, the API token will be removed from the local file storage only and the token stored in the remote server has to be manually revoked.

This is how the command is used:

```
$ terraform logout <HOSTNAME>
```

In case `HOSTNAME` is not specified, it will default to the HCP Terraform host at `app.terraform.io`.

The console Command

The `terraform console` command is used to start an interactive console to try out and experiment with expressions. For example, if you would like to try out expressions or any functions before using them in the actual configuration, this console-based command will help. This command has two ways of operating based on the state file content.

If the state file is empty, you can use this command as usual to try out the expressions; if the state file is not empty and you already have the configuration in place, the command will place a lock on the state file during the operation.

For configurations such as the local backend, the `-state` command-line option can be used to point to a different state file than the `terraform.tfstate` file in the current working directory.

Shortcuts such as Ctrl + C or Ctrl + D can be used to exit the console, and the `exit` command can also be used.

This is how the command is used:

```
$ terraform console <console_options>
```

Look at a simple string function, `replace()` and `uuid()`, to print a **Universally Unique Identifier** (**UUID**) in the `terraform console` command:

```
$ terraform console
> replace(«ec2_instances», «_», «-»)
"ec2-instances"
```

```
> uuid()
```

```
"b00bdca8-7e54-0f08-7127-3184926735f7"
```

The output Command

The `terraform output` command is used to extract the values of the output variables from the Terraform state file.

This is how the command is used:

```
$ terraform output <output_options> OUT_VARNAME
```

If `OUT_VARNAME` is not specified, the command will extract all the output values, and when specified, it will extract the value of that output variable.

The following options are supported by the `terraform output` command to interact with the output values stored in the state file:

- `-json`: This option produces the output in JSON format.
- `-raw`: This option can be used to produce the output values in string format without any special formatting and it helps when the Terraform execution is integrated with shell scripts. This option only supports string, number, and Boolean values; complex data types will not work.
- `-no-color`: This option is used to disable color codes in the output.
- `-state=<PATH_TO_STATE_FILE>`: This is used to point to a state file in a different location and is not applicable for remote backends.

Another important consideration with this command is that the command will display any sensitive value in the state file as plain text when using the `-json` or `-raw` option and should be used cautiously to avoid exposing sensitive data.

Take a look at the `output.tf` file in this configuration that has a couple of output values, such as the access key ID and the **Amazon Resource Name** (**ARN**) of the IAM user the script creates:

```
output "access_key" {
  value = aws_iam_access_key.user_access_keys.id
}

output "user_arn" {
  value = aws_iam_user.test_user.arn
}
```

After the Terraform configuration is applied, you can try the `terraform output` command, which will show the values from the state file. A few examples follow:

Example 1 shows `terraform output` with no output variable names:

```
$ terraform output
access_key = "AKIAEXAMPLETESTVALUE"
user_arn = "arn:aws:iam::123456789012:user/test/test-aws-user"
```

Example 2 shows `terraform output` in JSON format:

```
$ terraform output -json
{
  "access_key": {
    "sensitive": false,
    "type": "string",
    "value": "AKIAEXAMPLETESTVALUE"
  },
  "user_arn": {
    "sensitive": false,
    "type": "string",
    "value": "arn:aws:iam::123456789012:user/test/test-aws-user"
  }
}
```

Example 3 shows `terraform output` with an output variable name:

```
$ terraform output user_arn
"arn:aws:iam:123456789012:user/test/test-aws-user"
```

The show Command

The `terraform show` command is used to read the state file or the plan output file and show the output in human-readable format. This command can also be used in automation cases with the `-json` option flag.

This is how the command is used:

```
$ terraform show <show_options> <STATE_OR_PLAN_FILE>
```

If `STATE_OR_PLAN_FILE` is not specified, the state file in the default location will be referenced.

The following command options are supported by the `terraform show` command:

- `-json`: This option produces the machine-readable JSON output.
- `-no-color`: The output will not have any color if this option is specified.

Like the `terraform output` command, sensitive information will be displayed in plain text if the `-json` option is specified.

The graph Command

The `terraform graph` command is used to create a graph with the dependency relationships of the resources and data blocks for a given configuration as per the conventions in the **DOT language**. The DOT language saves the graph with `.gv` or `.dot` extensions and these are outside the scope of this book.

By default, the `graph` command results in a simple graph of resources and dependencies, and with the `-type` option, different types of graphs can be created with more detail and are helpful when you work with complex configurations.

This is how the command is used:

```
$ terraform graph <graph_options>
```

The supported command options are listed here:

- `-plan=tfplan`: This option creates the graph for the given plan that implies `-type=apply`.
- `-draw-cycles`: This option can be used to highlight the cycles in the graph with colored edges, and this is helpful for debugging any cycle errors.
- `-type=<operation_type>`: This option is used to specify the operation type for which the graph has to be created. The possible values are `plan`, `apply`, `plan-refresh-only`, and `plan-destroy`.

For the sample configuration you have in this chapter, the `terraform graph` command produces the following output. Please note that the command needs the Graphviz tool to be installed on your local workstation. Here is the download link: `https://graphviz.org/download/`

Here is an example command:

```
$ terraform graph | dot -Tpng > ch4_graph.png
```

Figure 4.1 shows the resource dependencies in graph format:

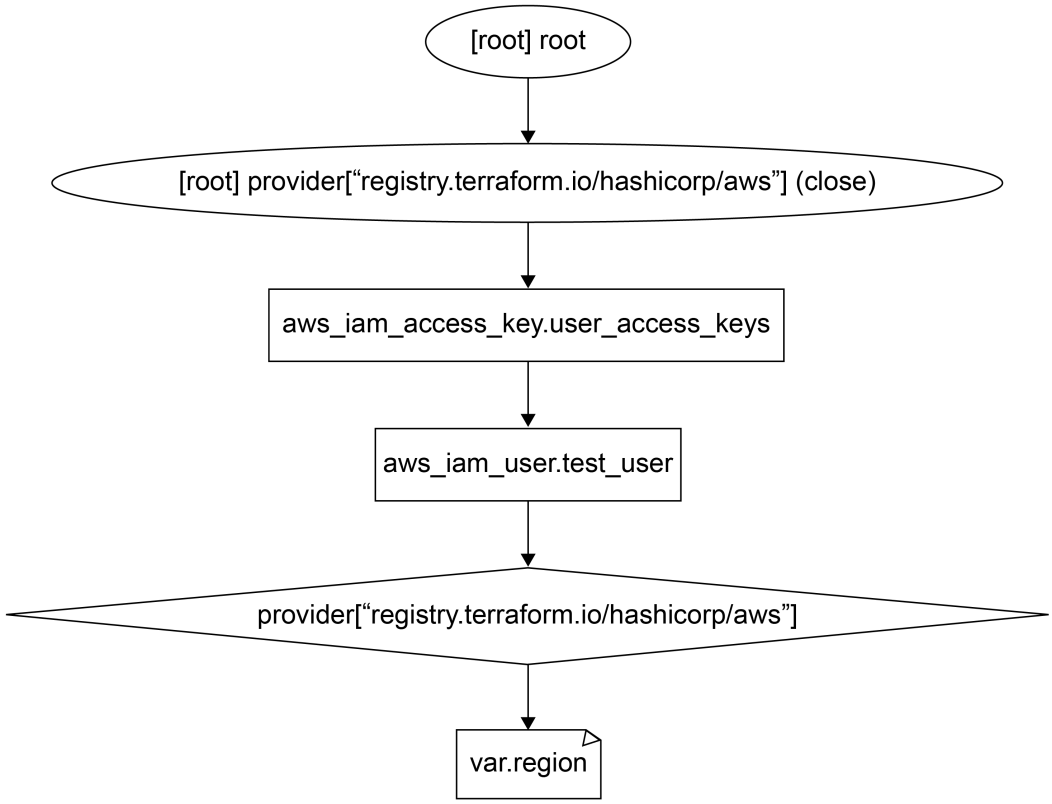

Figure 4.1: The graph command output

The import Command

The `terraform import` command is used to import the existing real-world infrastructure resources into your configuration state so it can be managed by Terraform.

The `import` command expects a resource configuration block to be written manually for the resource and then you can run the `import` command in Terraform CLI. This command itself will not generate any configuration.

If you would like Terraform to generate the configuration, the `import` block functionality can be used (available with Terraform v1.5.0 or later). The `terraform import` command can import only one resource at a time and the `import` block can import more than one resource at the same time.

This is how the command is used:

```
$ terraform import <import_options> RESOURCE_ADDRESS RESOURCE_ID
```

This command will import the resource with the specified Resource ID at the given Resource Address. If the resource is defined inside the module, the module name with the keyword should be appended before the resource address.

Here is an example:

```
$ terraform import module.foo.aws_s3_bucket.testbucket test-bucket-
qwi02
```

The resource ID can vary based on the resource type you intend to import. In the example configuration you are using in this chapter, the resource ID refers to the access key ID in the case of IAM access keys. So, if you are trying to import an IAM access key resource into Terraform, the `import` command would look like this:

```
$ terraform import aws_iam_access_key.user_access_keys
AKIAEXAMPLETESTVALUE
```

Take a look at the options available for this command:

- `-config=<PATH>`: This option can be used to specify the alternate location that contains the configuration scripts. If not specified, Terraform will look for configuration scripts in the current working directory.
- `-input=true`: Setting this option will ask for input to execute the `init` command if necessary. If this is set to `false`, it will cause an error if the input is not provided.
- `-lock=false`: This option will disable the locking of state files.
- `-lock-timeout=<duration>`: This option will override the duration of the state lock with the user-provided value. The default is zero seconds. It might help in scenarios where there are parallel processes running pointing to the same state file and it makes sense to wait for the lock to get released for the next process to use it.
- `-no-color`: This option is used to disable color codes in the output.
- `-parallelism=<COUNT>`: This option can be used to override the maximum number of parallel operations on the machine running Terraform.
- `-var "VARIABLE_NAME=VALUE"`: Use this option to set a value for the variable declared in the main configuration. If there are multiple variables, the option can be used for each variable name.
- `-var-file=<FILENAME>`: This option is used to set the values of multiple input variables, usually using the `*.tfvars` file. The option can also be used multiple times to pick the values from more than one file.

- `-provider=provider`: This option is used to override the default provider configuration while importing the object. By default, Terraform uses the provider specified in the configuration for the target resource. This option is deprecated.

> **Note**
> For configurations using the local backend only, `terraform import` also accepts the legacy options of `-state`, `-state-out`, and `-backup`. For more details on these options, please refer to the **Options** section under the `terraform apply` command.

State Management and the terraform state Command

As you have, the Terraform state file, `terraform.tfstate`, is critical to the core workings of a Terraform tool. The individual entries in the Terraform state file map directly to real-world infrastructure. This allows Terraform to sync the resources whenever there are changes to the configuration.

The state file is managed and updated automatically when commands such as `plan` and `apply` are executed. Sometimes, there might be a need to interact with the state file and make some adjustments to accommodate the changes made outside Terraform. This is where the `terraform state` command plays a vital role.

You can start with the `terraform state` command and the options associated with it. There are also subcommands within the `state` command that help in different scenarios, which will be discussed further in this topic.

The state Command

As discussed previously, the `terraform state` command is used mainly for state management. This command will modify the state file seamlessly without you having to work directly with the state file and editing it.

This is how the command is used:

```
$ terraform state [subcommand] <state_options> <ARGUMENTS>
```

The `terraform state` command will work the same way with both the local and remote backends. In the case of a local backend, the command will run faster, as there are no network hops, and remote backends take some time and the turnaround time will be delayed. Remote backends have their own advantages, such as security, backup management, and restricted access control.

The `state` command output can also be integrated with other tools, such as `jq`, and can be automated. The subcommands will also take a backup of the state file before it is modified by them. There are also subcommands such as `terraform state list` and `terraform state show` that are read-only and will not take any backups.

Subcommands of terraform state

The following section will cover the subcommands of the `terraform state` command; each has its own use case that will be discussed in detail.

The terraform state list Command

The `terraform state list` command will list all the resources within the state file.

This is how the command is used:

```
$ terraform state list <list_options> <RESOURCE_ADDRESS>
```

If `RESOURCE_ADDRESS` is not specified, the command will list all resources available in the state file. When specified, the command will list only the resources matching the criteria.

There could be situations where you might be dealing with a large number of resources and listing everything in one go may not be required. In that case, you can choose to list only specific resources, all resources belonging to a specific module, all instances created by the same resource group, and so on.

The options supported by this command are as follows:

- `-state=<PATH>`: This option is used to specify the state file to be considered. If not provided, it will default to the `terraform.tfstate` file in the current working directory.
- `-id=<ID>`: This option is used to filter the resource by ID.

For a better understanding, a few examples are provided here:

Example 1 lists all resources in the state file:

```
$ terraform state list
Output:
aws_iam_access_key.user_access_keys
aws_iam_user.test_user
```

Example 2 lists resources by resource address:

```
$ terraform state list aws_iam_user.test_user
Output:
aws_iam_user.test_user
```

In this case, if multiple instances of the same resource type are provisioned, all of them will be listed with the index. In your case, you only have one, so it displays the same resource.

Example 3 filters resources by resource ID:

```
$ terraform state list -id=AKIAEXAMPLETESTVALUE
Output:
aws_iam_access_key.user_access_keys
```

The terraform state show Command

The `terraform state show` command is used to display the attributes of a single resource in the state file.

This is how the command is used:

```
$ terraform state show RESOURCE_ADDRESS
```

In this command, `RESOURCE_ADDRESS` is not optional, and hence, needs to be provided always. Upon successful execution, the attributes of the matching resource will be displayed.

The resource address format can be in any valid form that can be accepted by Terraform, and the supported command option is as follows:

`-state=<PATH>`: This option is used to specify the state file to be considered. If not provided, it will default to the `terraform.tfstate` file in the current working directory.

You can now run this command with the available configuration:

```
$ terraform state show aws_iam_user.test_user
Output:
# aws_iam_user.test_user:
resource "aws_iam_user" "test_user" {
    arn           = "arn:aws:iam::123456789012:user/test/test-aws-
user"
    force_destroy = false
    id            = "test-aws-user"
    name          = "test-aws-user"
    path          = "/test/"
    tags          = {
        "createdby" = "terraform"
    }
    tags_all      = {
        "createdby" = "terraform"
```

```
    }
    unique_id     = "AIDAEXAMPLEUSERIDKHNJ"
}
```

You can see that this command lists all the possible attributes for the given resource from the state file and it is meant for human interaction. If there are any automation scenarios, the `terraform show -json` command can be used to get the same output in machine-readable JSON format, which we already discussed.

The terraform state mv Command

The default behavior of Terraform is that if you change the configuration after provisioning the resources already, it will track the changes and ensure that the remote objects match the current configuration settings.

How does this work? During the `apply` stage, Terraform destroys the resource provisioned with the previous snapshot of the configuration and a new resource will be provisioned with the latest changes.

But there will be rare scenarios where you do not want to destroy the remote object that's already active and want to track it with the new resource address. This command will help in this case.

This is how the command is used:

```
$ terraform state mv <move_options> SOURCE DESTINATION
```

Even here, `SOURCE` and `DESTINATION` should follow the resource-addressing conventions. When valid values are provided, Terraform will look at the specific resource address and see whether the object exists. If successful, it will move the resource to the new destination resource address.

The options supported are as follows:

- `-dry-run`: This option will report all resources in the state matching the source address but will not move/make any changes. It can be used for testing purposes before making the actual change.
- `-lock=false`: This option will disable the locking of state files; the default behavior is state locking will be enabled.
- `-lock-timeout=<duration>`: This option will override the duration of the state lock with the user-provided value and the default is zero seconds. It might help in scenarios where there are parallel processes running pointing to the same state file and it makes sense to wait for the lock to get released for the next process to use it.

A simple example to understand this command would be renaming the IAM user resource in the current configuration as follows.

This is the code before changes:

```
resource "aws_iam_user" "test_user" {
  name = "test-aws-user"
  path = "/test/"
. . .
}
```

This is the code after changes:

```
resource "aws_iam_user" "preprod_user" {
  name = "test-aws-user"
  path = "/test/"
. . .
}
```

In this case, the `terraform state mv` command will help us track the IAM user resource under the new address. The command to be used is as follows:

```
$ terraform state mv aws_iam_user.testuser aws_iam_user.preprod user
```

> **Note**
> For configurations using the local backend only, the `terraform state mv` command also accepts the legacy options of `-state`, `-state-out`, and `-backup`.

The terraform state rm Command

The `terraform state rm` command will help remove the remote object from the state file so it can continue to exist without destroying it. This also means that if the same configuration is reapplied, the new resources will be created again for the removed instances to be in sync.

This is how the command is used:

```
terraform state rm <remove_options> RESOURCE_ADDRESS
```

The supported command options are the same as the `terraform state mv` command:

- `-dry-run`: This option will report all resources in the state matching the source address but will not move any resources/make a change. It can be used for testing purposes before making the actual change.
- `-lock=false`: This option will disable the locking of state files.

- `-lock-timeout=<duration>`: This option will override the duration of the state lock with the user-provided value and the default is zero seconds. It might help in scenarios where there are parallel processes running pointing to the same state file and it makes sense to wait for the lock to get released for the next process to use it.

> **Note**
> For configurations using the local backend only, the `terraform state rm` command also accepts the legacy options of `-state`, `-state-out`, and `-backup`.

The terraform state replace-provider Command

If there are scenarios to replace the provider currently used in the configuration with the new provider configuration, the `terraform state replace-provider` command can be used. This command will also take a backup of the state file before the update and the backup option cannot be disabled.

This is how the command is used:

```
$ terraform state replace-provider FROM_PROVIDER TO_PROVIDER
```

Upon successful execution, resources from `FROM_PROVIDER` will be moved to `TO_PROVIDER`. The supported options are listed here:

- `-auto-approve`: This option will execute the changes without waiting for any approvals.
- `-lock=false`: This option will disable the locking of state files.
- `-lock-timeout=<duration>`: This option will override the duration of the state lock with the user-provided value and the default is zero seconds. It might help in scenarios where there are parallel processes running pointing to the same state file and it makes sense to wait for the lock to get released for the next process to use it.

> **Note**
> For configurations using the local backend only, the `terraform state replace-provider` command also accepts the legacy options of `-state`, `-state-out`, and `-backup`.

The terraform state pull Command

The `terraform state pull` command downloads the state file from its current location, upgrades it so that it is compatible with the locally installed Terraform, and prints the raw output to the console.

This is how the command is used:

```
$ terraform state pull
```

This command can also be used in cases where the information from the state should be retrieved when paired with other third-party tools such as `jq`. It is not recommended to use this command with the state file stored in the remote backend.

The terraform state push Command

The `terraform state push` command is used to manually upload a local state file to the remote backend and should be used very rarely, when there are no other alternatives.

This is how the command is used:

```
$ terraform state push <push_options> PATH
```

The state file from the specified path will be pushed to the currently configured backend.

Before the state file is pushed, Terraform performs safety checks to ensure the changes are completely secure. To disable the checks, the `-force` option can be used but is not recommended.

> **Note**
> To use the `state pull` and `state push` commands, Terraform expects the file to be in UTF-8 format without the **byte order mark** (**BOM**).

Summary

In this chapter, you read about the core Terraform workflow commands. This was followed by explanations of special-purpose commands such as `console`, `graph`, and `import`, used as and when the need arises. The `terraform state` command was then discussed in detail with the subcommands. You also looked at how each command helps the developer interact with and manage the state file beyond infrastructure provisioning.

Now, you can proceed with the next chapter, on how to use the commands with Terraform CLI with full confidence. This will help in increasing productivity and the seamless transition to advanced concepts such as Terraform modules, HCP Terraform, and so on.

Exam Readiness Drill – Chapter Review Questions

Apart from a solid understanding of key concepts, being able to think quickly under time pressure is a skill that will help you ace your certification exam. That is why working on these skills early on in your learning journey is key.

Chapter review questions are designed to improve your test-taking skills progressively with each chapter you learn and review your understanding of key concepts in the chapter at the same time. You'll find these at the end of each chapter.

How to Access these Resources

To learn how to access these resources, head over to the chapter titled *Chapter 11*, *Accessing the Online Practice Resources*.

To open the Chapter Review Questions for this chapter, perform the following steps:

1. Click the link – `https://packt.link/HCorp003Ch4`.

   Alternatively, you can scan the following **QR code** (*Figure 4.2*):

Figure 4.2 – QR code that opens Chapter Review Questions for logged-in users

2. Once you log in, you'll see a page similar to the one shown in *Figure 4.3*:

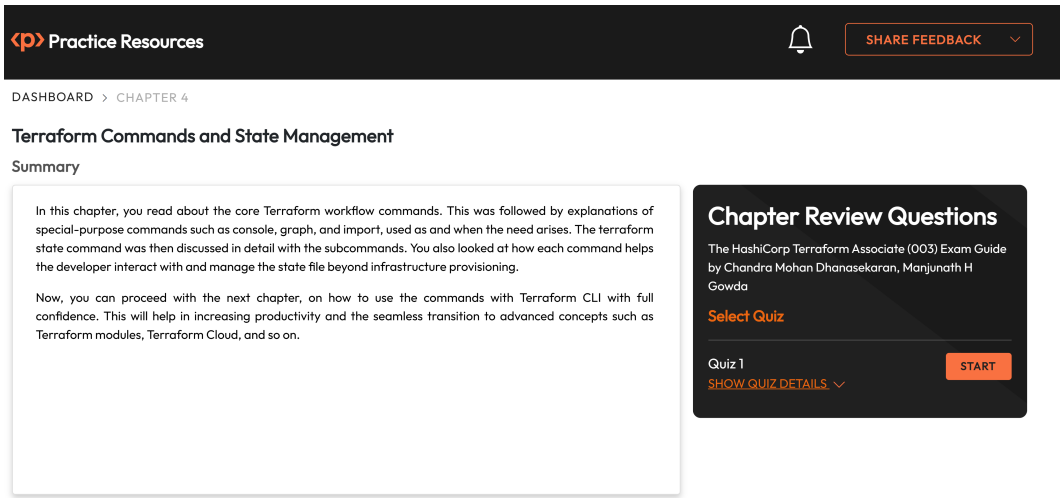

Figure 4.3 – Chapter Review Questions for Chapter 4

3. Once ready, start the following practice drills, re-attempting the quiz multiple times.

Exam Readiness Drill

For the first three attempts, don't worry about the time limit.

ATTEMPT 1

The first time, aim for at least **40%**. Look at the answers you got wrong and read the relevant sections in the chapter again to fix your learning gaps.

ATTEMPT 2

The second time, aim for at least **60%**. Look at the answers you got wrong and read the relevant sections in the chapter again to fix any remaining learning gaps.

ATTEMPT 3

The third time, aim for at least **75%**. Once you score 75% or more, you start working on your timing.

> **Tip**
> You may take more than **three** attempts to reach 75%. That's okay. Just review the relevant sections in the chapter till you get there.

Working On Timing

Target: Your aim is to keep the score the same while trying to answer these questions as quickly as possible. Here's an example of how your next attempts should look like:

Attempt	Score	Time Taken
Attempt 5	77%	21 mins 30 seconds
Attempt 6	78%	18 mins 34 seconds
Attempt 7	76%	14 mins 44 seconds

Table 4.1 – Sample timing practice drills on the online platform

> **Note**
> The time limits shown in the above table are just examples. Set your own time limits with each attempt based on the time limit of the quiz on the website.

With each new attempt, your score should stay above **75%** while your "time taken" to complete should "decrease". Repeat as many attempts as you want till you feel confident dealing with the time pressure.

5
Terraform Modules

Imagine that your company's **Quality Assurance** (**QA**) team needs a test environment to be spun up in a very short interval and the dependency on your team is causing delays.

The best way to handle this is to facilitate a self-service option for the QA team where they can create the required environment themselves by passing some unique inputs for each environment. This can be achieved using a module.

Modules are one of the core concepts of Terraform because of their extensive use in the production environment. You will rarely find a production environment where modules are not used. A good understanding of modules is required not just from a certification perspective but also for day-to-day operations while managing a Terraform-based setup. The following are the key topics that will be covered in this chapter:

- Why do we need modules?
- What is a module in Terraform?
- Advantages of modules
- Types of modules
- Module structure
- Module syntax
- Key points to consider before creating modules
- Key points to consider while using a module
- Drawbacks of modules

Once you complete these topics, you will find exercises where you will learn how modules are used in the real world in a practical way:

- Using a root module to provision resources
- Using a local module to provision resources
- Using a remote module to provision resources

By the end of this chapter, you will have a good understanding of modules and some of the key considerations from a design point of view. Operationally, you will also be able to write your own modules for any requirement and will be able to use a public module and customize it for your needs.

Technical Requirements

This chapter has three exercises. The following are the pre-requisites for you to get started with the exercises:

- An AWS account and an IAM user with required permissions (or an SSO user if your organization is using AWS SSO). Refer to the following URL for steps on how to create an IAM user in AWS: `https://docs.aws.amazon.com/IAM/latest/UserGuide/id_users_create.html`.
- The AWS CLI installed (version 2.0 and above preferred). For detailed instructions on installing the AWS CLI, go to `https://docs.aws.amazon.com/cli/latest/userguide/getting-started-install.html`.
- Terraform installed (version 1.4.0 and above preferred).
- The code required for the exercise is present in our GitHub repository.
- Here is the URL for the GitHub repository: `https://github.com/PacktPublishing/Hashicorp-Certified-Terraform-Associate-003-Exam-guide-Second-Edition/tree/main`
- Clone the GitHub repository to your local machine by running the following command:

```
git clone https://github.com/PacktPublishing/Hashicorp-Certified-Terraform-Associate-003-Exam-guide-Second-Edition.git
```

- Navigate to *Chapter 5* by running the following command:

```
cd Hashicorp-Certified-Terraform-Associate-003-Exam-guide-Second-Edition/ch5
```

- You should find the `README.md` file along with three folders, each of which contains an exercise.
- Refer to `README.md` for instructions on the exercises.

> **Note**
> The exercises launch real resources in the AWS cloud. Almost all the resources are free except for the EC2 instance, which gets charged till you delete it. It should not cost you more than a dollar if you complete the exercise within three to four hours. Please make sure you delete the resources to avoid getting charged.

Why Do We Need Modules?

When you start using Terraform, you start by creating a few infrastructure objects by writing code in a single configuration file. As your requirements grow, you start adding additional configurations to the same file or adding new files to the same directory. Theoretically, you can even keep using this approach. However, is this the right approach that is recommended by HashiCorp?

Before answering this question, take a look at the drawbacks of this approach:

Complexity

When you start copy-pasting the code for similar requirements or multiple resources of the same configuration, the complexity (imagine having 50 blocks of code to create 50 instances) will increase with the growth in the number of resources. It becomes very difficult for the team managing Terraform to understand the code and navigate it for either changes or troubleshooting.

Duplication of Code

You use the same block of code with minor changes for similar resources. One of the key principles of software development is **Don't Repeat Yourself** (**DRY**). When you are trying to adopt the best practices of software development in **Infrastructure as Code** (**IaC**), duplicating the code goes against one of the important principles of software development. For example, if you want to launch three EC2 instances of the same configuration but with different names, you end up copying the same code thrice and changing a single argument for the **name**.

Segregation

The segregation of different layers of infrastructure resources is important to assign ownership to different teams. Imagine a situation where the network team wants to create code for all the networking requirements and the application team wants to write code for app-related requirements. How are you going to segregate and delegate the ownership?

Misconfiguration

When you want to make changes, apart from knowing what exactly the Terraform command does, you also need to know the whole environment to gauge the impact a simple change will have on your setup. The chance of a misconfiguration and its impact on the environment is very high.

So, how can you solve these problems? What is the recommendation from HashiCorp? What is being followed by the industry? Terraform modules are the answer. By the end of this chapter, you will understand how Terraform solves all these problems.

What Is a Terraform Module?

In non-technical terms, a Terraform module is a template using which you can launch logically grouped resources as per your requirement even without knowing its inner workings.

You define the template once and use it any number of times to launch resources. For example, you can create a module for the launch of a test environment of your product and your QA team can use this module any number of times to launch the test environment.

HashiCorp calls the module a set of configuration files in a single directory that helps you organize and re-use Terraform configuration. In other words, any directory with Terraform configuration files to create a certain resource(s) can be called a module.

Terraform uses modules to package resource configurations, and the user can reuse these modules whenever there is a requirement to launch resources packaged in a module.

Advantages of Modules

In this section, you will look at some advantages of modules:

Reduces Complexity

Modules group logical components, making them easy to manage and update when required. This makes it easy for the team that manages Terraform to understand the code and make the required changes.

Reduces Code Duplication

You write modules once (or use public modules) and these get used to launch resources. There is no duplication of code here. You only pass values to the variables to decide the number of resources or the configuration of the resources.

This not only prevents the duplication of code but also saves time by reducing the effort required for the same task.

Segregation

Modules inherently support segregation because you can build one layer on top of another. A root module is the place where a child module gets referenced in one of the configuration files. The child module variable's value gets passed and the required outputs are defined. The output of one module can be used as input to another resource's configuration. Hence, you can create a separate layer for each infrastructure tier. For example, in a typical three-tier architecture, there will be a web layer (presentation layer), an app layer, and a database layer.

To deploy this three-tier-based application, you also need a network layer, which is managed by the network team.

The network team will be responsible for creating the networking components that will be used by other teams to deploy their respective layers. The app/web/DBA team will have their own modules for the launch of respective resources. They will own these modules but when they launch the infrastructure objects, they will consume the outputs of the networking layer to make sure these run within the defined network.

Reduces Misconfigurations

It is only human to err. You have likely experienced a situation where you or one of your colleagues made changes in an environment/file/resource that were not meant to be made. When you use individual files, you define every single parameter, and hence, the chance of human error is higher. With modules, you standardize most of these configurations and only allow a few important configurations to be modified via variables. This automation helps in reducing the chance of misconfigurations.

Self-Service

If you want to allow people to create resources on their own without spending too much time learning about Terraform, modules can help. You can just publish a module and the variables whose values need to be passed by the respective team to create the resource.

All the complex configurations are hidden under the hood. The end user just needs to understand the inputs and outputs. Inputs can be passed as input variables and the outputs of the modules need to be accessed using standard syntax.

Apart from these key advantages, modules help in quick deployments and maintain a consistent environment where the best practices of the industry can be tested once and deployed to environments via modules.

Types of Modules

Depending on the criteria one uses, modules can be classified in multiple ways. *Figure 5.1* shows the classification of modules based on two important criteria:

- Based on the entry point for the module (i.e., where `terraform plan/apply` is run):
  - Root module
  - Child module
- Based on where the configuration files are stored/installed:
  - Local module
  - Remote module

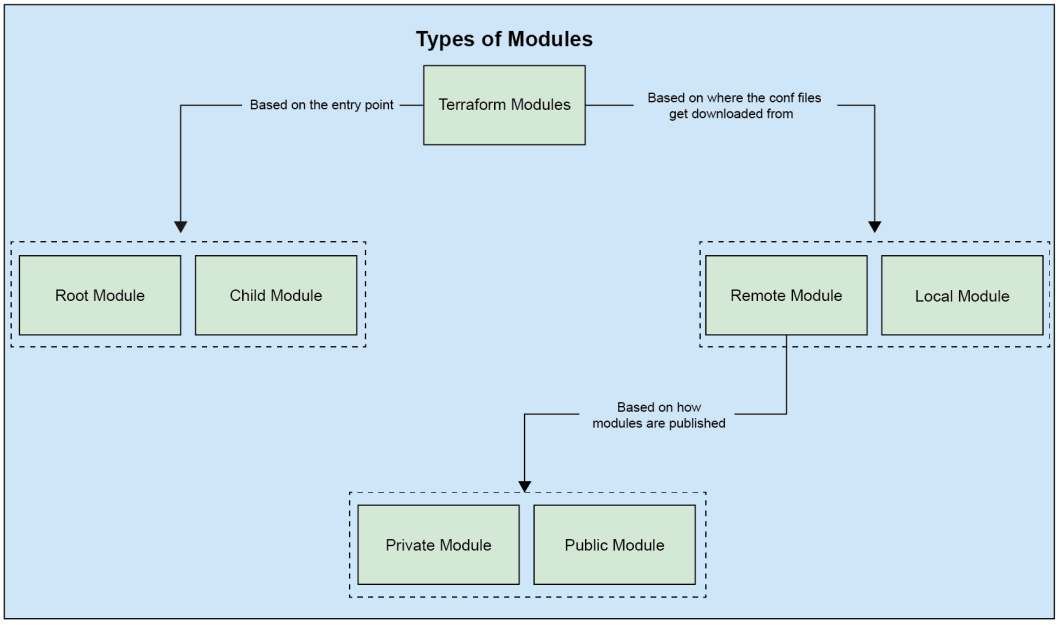

Figure 5.1: Types of modules

Now that you have seen the types of modules, it is time to take a look at each module type in detail in the following section.

The Root Module

Any folder with a set of Terraform configuration files to create infrastructure objects can be called a root module.

If the Terraform module is a template, someone must instantiate it, that is, make a copy of the infrastructure objects using the template. This is where the root module comes in.

The root module is the primary entry point for the module. This is the directory with the configuration files where the user will run `terraform plan` or `terraform apply`.

A root module can have a few configuration files to create the infrastructure and/or it can call the child modules as shown in *Figure 5.2*.

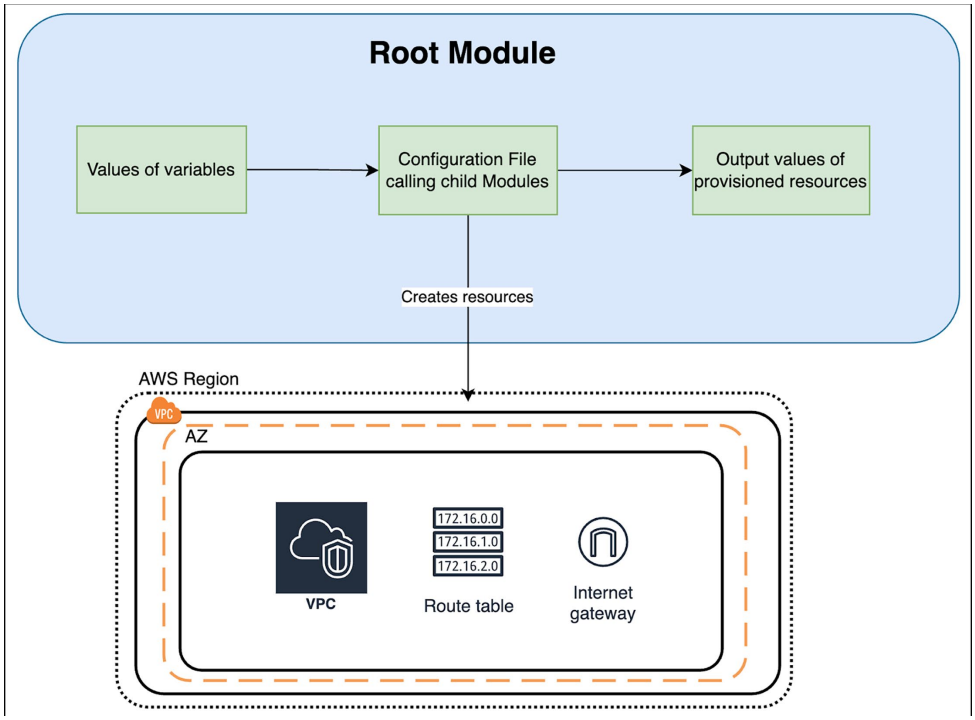

Figure 5.2: Root module

A typical root module's files are shown in *Figure 5.3*.

When you call the child module from the current directory (i.e., the root module), it loads the configuration files of the child module and replaces the variable value with the one you defined in the current directory.

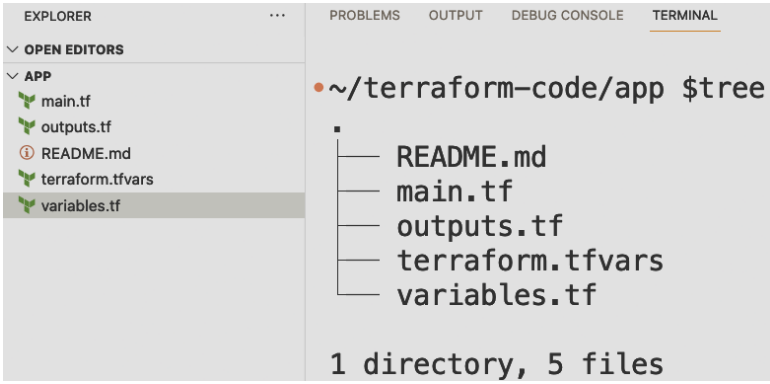

Figure 5.3: Root module file structure example

The Child Module

A module is called a child module when it gets called by the root module using a module block, as shown in *Figure 5.5*.

Once you have created the child module, you can call it any number of times within the configuration files in your root module. It can also be called from multiple root modules, provided they have access to this child module. When you run `terraform get` or `terraform init` from the root module, the child module gets downloaded if it's a remote module. If it is a local module, the `modules.json` file under `.terraform/modules` will contain the path of the modules in the local filesystem.

> **Note**
> When people talk about modules, most of the time it is in the context of a child module. If it is a root module, it is normally specified.

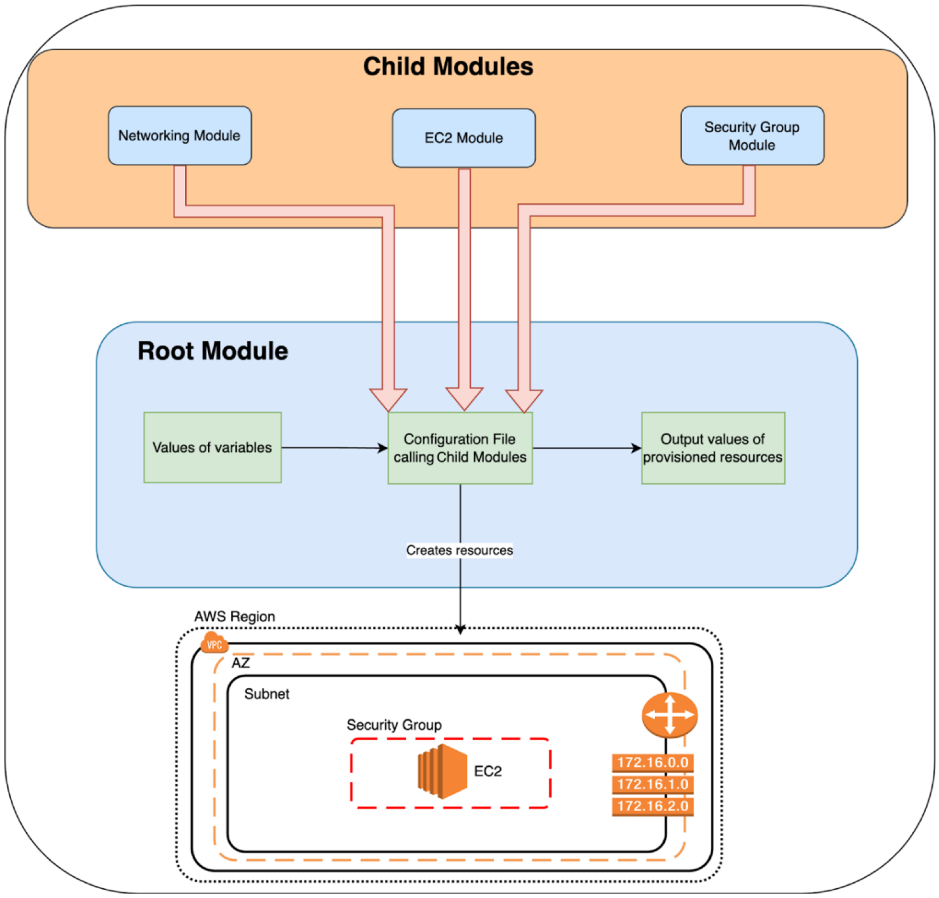

Figure 5.4: Child module called by root module

While *Figure 5.4* is a pictorial representation of the child module's work, the following *Figure 5.5* shows the code/file representation of the module.

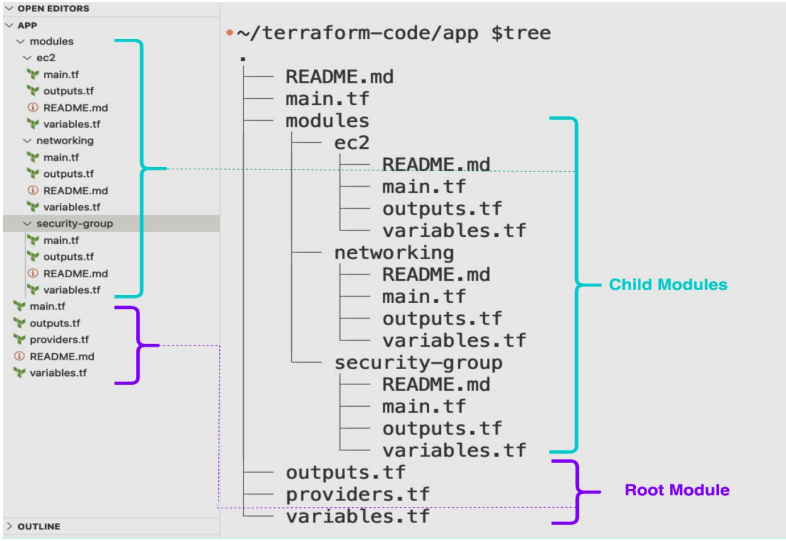

Figure 5.5: Child module and root module file structure example

> **Note**
> Some child modules have their own child modules. Such a module is called a nested module.

Local Modules

As explained earlier, a module is a bunch of configuration files and needs to be stored somewhere before it gets called by anyone who wants to use it.

If you store these configuration files on your local filesystem, it is called a local module.

When you are calling a local module, it must begin with `./` or `../` to indicate that the module is present in the local filesystem, as shown in *Figure 5.6*.

HashiCorp recommends using the relative filesystem paths to refer to the local Terraform modules instead of the absolute filesystem paths. Using the absolute path tightly couples the configuration file path to the specific computer's filesystem layout.

You will be able to understand this well with the following example, where you will try both the full path and relative path to call the child module from the root module.

This is the child module's path:

```
/Users/manju/Desktop/terraform/vpc-module
```

This is the root module where the execution is happening:

```
/Users/manju/Desktop/terraform/example
```

Within the `main.tf` file of the root module, you will call the child module.

If you use the full path as follows, your code will not be able to run on your colleague's laptop as the path where they run Terraform will be different:

```
module "vpc"{
     source = "/Users/manju/Desktop/terraform/vpc-module"
     ---- Truncated ----
}
```

Instead, you should use a relative path, as follows. This should work on any computer provided the operating system is Linux:

```
module "vpc"{
     source = "../vpc-module/"
     ---- Truncated ----}
```

```
###  CREATING VPC, SUBNET, ROUTE TABLE, IGW ###
module "vpc" {
  source            = "./modules/networking"
  vpc_cidr_block    = "10.51.0.0/16"
  vpc_name          = "child-module-vpc"
  subnet_cidr_block = "10.51.1.0/24"
  az                = "ap-south-1a"
  igw_name          = "child-module-igw"
}

###  CREATING SECURITY GROUP AND RULES ###
module "security_group" {
  source              = "./modules/security-group"
  vpc_id              = module.vpc.vpc_id
  security_group_name = "child-module-sg"
  allowed_ip          = "0.0.0.0/0"
  from_port           = "22"
  to_port             = "22"
}
```

source path is within the local filesystem

Figure 5.6: Example code for local module implementation

While *Figure 5.6* shows a local child module's path, *Figure 5.7* illustrates the local child module's initialization.

```
~/terraform-code/app $terraform init

Initializing the backend...
Initializing modules...
- ec2_instance in modules/ec2
- security_group in modules/security-group
- vpc in modules/networking

Initializing provider plugins...
- Finding hashicorp/aws versions matching "5.34.0"...
- Installing hashicorp/aws v5.34.0...
- Installed hashicorp/aws v5.34.0 (signed by HashiCorp)

Terraform has created a lock file .terraform.lock.hcl to record the provider
selections it made above. Include this file in your version control repository
so that Terraform can guarantee to make the same selections by default when
you run "terraform init" in the future.

Terraform has been successfully initialized!
```

Local child modules getting initialized

Figure 5.7: Local child modules getting initialized

When installing a local module, Terraform will not download the module, but will instead refer to the source directory using the path of the modules as specified in the `modules.json` file under `.terraform/modules`.

Storing the configuration on a local filesystem limits the ability to share the modules and work as a team. But this works great for testing modules before pushing them to a version control system or the Terraform Registry to make it a remote module.

Remote Modules

If a module's configuration files are stored and downloaded from a remote system like the Terraform Registry, a version control system (for example, GitHub, Bitbucket, etc.), HTTP URLs, an S3 bucket, or a GCS bucket, then these are called remote modules.

When installing a remote module, Terraform will download the module into the `.terraform` directory in your configuration's root directory. Any changes in the remote module will be reflected only after you upgrade the modules by running the `terraform init -upgrade` command.

A remote module can be public (Terraform Registry) or private (HCP Terraform private registry).

The Terraform Registry is the native way of distributing modules. It is integrated directly into Terraform, so a Terraform configuration can refer to any module published in the registry using the `<NAMESPACE>/<NAME>/<PROVIDER>` syntax: for example, `hashicorp/consul/aws`.

A registry is an index of modules and is the easiest way to find public modules that are created by the community.

Unless you specify a specific version using the **ref** argument, Terraform will choose the default branch of the selected repository. If you want to use a different version, you can use the **ref** argument with the tag name, branch, or SHA-1 hash.

Remote modules can be further divided into two, depending on the availability of the modules to the user:

- Private modules
- Public modules

Private Modules

Private modules are modules that are available only to members of your team/organization based on how you have set them up.

You can store your modules in the version control system and decide who has access to this repository. Another option is to use the organization's HCP Terraform private registry that is integrated with your version control system. Apart from keeping modules private, HCP Terraform offers other features that will be discussed in *Chapter 9, Understanding HCP Terraform's Capabilities*.

The private registry modules have the following syntax for their name:

```
terraform-<PROVIDER>-<NAME>
```

Examples are `terraform-gcp-vault` and `terraform-aws-vpc`.

Public Modules

Public modules are those that can be downloaded by everyone. The most popular place for hosting a public module is the Terraform Registry. A registry exposes some additional information that is very helpful in deciding whether you can use a module or should try others.

Some of these modules are uploaded by vendors themselves and are validated by HashiCorp. These are trusted to be good ones.

You can also see individuals/teams upload modules, allowing everyone to consume them.

An example of downloading a remote module from the Terraform Registry is shown in *Figure 5.8*:

```
module "ec2_instance" {
  source                  = "terraform-aws-modules/ec2-instance/aws"
  version                 = "5.6.0"
  name                    = "public-module-ec2"
  ami                     = data.aws_ami.root_module_ami.id
  instance_type           = "t3.micro"
  subnet_id               = module.vpc.public_subnets[0]
  vpc_security_group_ids  = [module.security_group.security_group_id]
  key_name                = aws_key_pair.root_module_kp.id
  tags = {
    Name = "public-module-ec2"
  }
}
```

source path is referring to the implied terraform public registry for child module

Figure 5.8: Example code for a public (and remote) module (Terraform Registry)

An example of downloading a remote module from GitHub's public repository is shown in *Figure 5.9*:

```
main.tf
1  module "example" {
2      source = "github.com/manju712/terraform-remote-module"
3      }
4
```

Source path is referring to the GitHub's repository.

Figure 5.9: Example code for public (and remote) module (GitHub)

You must always verify public modules before using them. Since anyone can contribute to the public modules on the Terraform Registry, you will not know whether the code is of high quality, free from security issues, or whether it follows best practices without verifying the code yourself. There is enough information about these modules that can help you decide whether you should use them. They are scattered between the Terraform Registry and GitHub.

Key parameters to verify a module in the Terraform Registry are the following:

- How old is the module?
- How active is the module? (How many times has it been downloaded in the last week/month?)
- Versions

Key parameters to verify a module in GitHub are the following:

- Is it getting frequent updates in terms of commits and versions?
- Does it have a good number of active contributors and stars?
- Are the license terms compatible with your organization's policies?
- Does it have pull requests and issues raised by the community? How has the response been for previous issues?
- Take a look at the code to rule out any undocumented resources or features.

Module Structure

There is no mandatory structure for modules. However, HashiCorp recommends following their standard module structure, since Terraform tools are designed to understand this structure in order to generate documentation and index modules when you upload modules to the Terraform Registry.

HashiCorp adds a root module as the mandatory component of the module structure. However, the focus here is mostly on the child module's structure. Hence, you will not see the root module as one of the requirements in the structure given here.

These are the key files that make up a module:

- `main.tf`: This is the main configuration file that defines the resources that need to be launched.
- `variables.tf`: Hardcoding the configuration value in the module's code makes it difficult to reuse it. You parameterize (define the variable but pass the value of the variable during execution) the arguments of the resources as variables and pass the value of these variables when you call this module from the root module. All the variables corresponding to the module are defined in the `variables.tf` file.
- `README.md`: This is a helper file to be written in the Markdown format. This gives information to the user on what the module is used for and how the module should be used.
- `outputs.tf`: All the resources created by the module are encapsulated and the attributes cannot be accessed from the root module unless they are exported in the child module as outputs. This is typically defined in the `outputs.tf` of the child module.

For example, if a security group is created using a child module, the ID of the security group will be available in the root module only if it is exported as an output in the child module and is also defined as an output in the root module – preferably, in `outputs.tf`.

You can use these outputs as input to other configurations or modules.

You typically refer to output values as `module.MODULE_NAME.OUTPUT_NAME`.

Most modules follow the preceding structure, and it is recommended that you use at least these files in a module.

Figure 5.10 shows the module with the minimum number of files as per the standard structure recommended by HashiCorp:

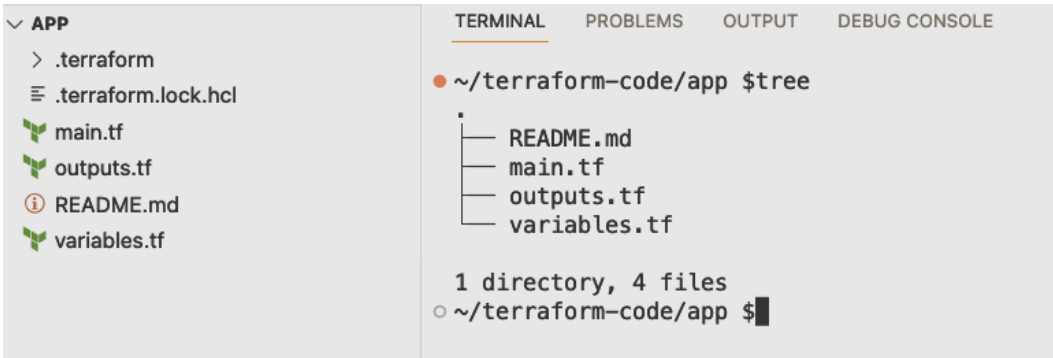

Figure 5.10: Module structure with minimal files

Some people also prefer to share more information about the module by adding additional files like `changelog.md` (key changes with version increment), folders like examples (with sample code for users), and modules with child modules. This is optional and you can add required files/folders based on your requirements.

The following URL is a good example for the module files. It has the `changelog.md`, `LICENSE`, and `README.md` file, along with `main.tf`, `variables.tf`, and `outputs.tf`:

`https://github.com/terraform-aws-modules/terraform-aws-vpc/tree/master`

The Module Block's Syntax

When you want to use child modules in the root module, there is a particular syntax you need to make use of. This syntax is part of the module block and is shown in *Figure 5.11*. Module blocks document the syntax for calling a child module from a parent module, including meta-arguments like `for_each`:

```
### GENERIC SYNTAX OF MODULE ###
module "local-name-for-the-module" {
  source    = "Local/Remote location of the child module"
  version   = "Version number of the module to be downloaded from the above source"
  VARIABLE1 = Value1
  VARIABLE2  = Value2
}

### EXAMPLE OF A MODULE ###
module "key_pair" {
  source  = "terraform-aws-modules/key-pair/aws"
  version = "2.0.2"

  key_name           = "my-keypair"
  create_private_key = true
}
```

Figure 5.11: Syntax and example of a module block

The name specified within `"  "` after the `module` keyword is considered the local name. This is used by other modules to refer to this module.

The content within { } is called the module body.

This module body contains four arguments:

- `source`: This is a mandatory argument. This can be a path of a local filesystem or a remote one (Terraform Registry, GitHub, and more). This has to be a literal string, and expressions or variable interpolations are not allowed here.
- `version`: This is used to constrain the version of the modules that can be downloaded. It is recommended to pin the version number in production. By default, the latest version gets downloaded when you pull the module. This can cause compatibility issues and break some functionalities. By pinning the versions, you can perform testing for the newer version separately, iron out issues, and then do an upgrade of this module. Here are the other aspects of this argument:
  - Terraform supports various operators (`=`, `!=`, `>`, `>=`, `<`, `<=`, `~>`) to help you pin versions.
  - `version` is not useful in the local modules as these are referenced directly in the local filesystem using the path specified in the `modules.json` file. Hence, the local modules used are always the latest ones.